Unidad 1:
INTRODUCCIÓN A LA PROGRAMACION ORIENTADA A OBJETOS (POO)

Elaborado por: Yolima S. Blanco Enamorado

Asesores: Ing. Mario R. Macea Anaya

DESARROLLO

PARADIGMAS DE PROGRAMACION

Un paradigma no es más que una ventana desde donde observamos, comprendemos, interpretamos e intervenimos el en mundo.

"Adam Smith define paradigma, en su libro 'Los poderes de la mente', como 'un conjunto compartido de suposiciones. Es la manera como percibimos el Mundo' […] los paradigmas de programación nos indican las diversas formas que, a lo largo de la evolución de los lenguajes, han sido aceptadas como estilos para programar y para resolver los problemas por medio de una computadora" (Instituto Tecnologico de Celaya)

Un paradigma de programación es considerado en palabras más sencillas como la forma, que establece las herramientas y el método que serán utilizados en la creación y construcción de un software por un programador.

Un lenguaje de programación, es una estructura de lenguaje formal que posee un conjunto de reglas, notaciones, símbolos y caracteres que le permiten a un programador poder expresar como es el proceso de datos y sus estructuras en una computadora.

Mediante los lenguajes de programación, el ser humano puede comunicarse con una computadora y decirle qué es lo que tiene que hacer y, al mismo tiempo, sirven para comunicarse con otros seres humanos y expresarles qué es lo que se quiere que haga una computadora.

La mayoría de los lenguajes de programación están basados en uno o más paradigmas, a continuación se explicaran algunos:

- **ENSAMBLADOR:** este paradigma se caracteriza por ser de bajo nivel, porque expresa las instrucciones de una forma más natural al hombre, pero mucho más natural y a la vez muy cercana al microcontrolador. Por tanto es el único lenguaje que entienden los microcontroladores; es el código máquina formado por ceros y unos del sistema binario.

  El lenguaje ensamblador trabaja con nemónicos, que son grupos de caracteres alfanuméricos que simbolizan las órdenes o tareas a realizar. La traducción de los nemónicos a código máquina entendible por el microcontrolador la lleva a cabo un programa ensamblador.

  El programa escrito en lenguaje ensamblador se denomina código fuente.

- **LOGICO:** este tipo de programación se basó en el uso de la lógica para el diseño y planteamiento de problemas, así como el control sobre las reglas que se deben tener en cuenta para alcanzar la solución. Es decir que nos ayuda o permite formalizar hechos del mundo real.

  La programación lógica, es trabajada en forma descriptiva, estableciendo relaciones entre entidades, indicando no como, sino que hacer.

  El lenguaje lógico por excelencia es **Prolog**, que puede ser a la vez declarativo.

- **FUNCIONAL:** esta programación está basada en el uso de funciones aritméticas que no maneja datos alterables; es enfática en la aplicación de funciones en contraste con el estilo de programación imperativa.

  La programación funcional busca tratar el problema de la programación desde el punto de vista matemático, utilizando las funciones como base para la construcción de los algoritmos y estructuras de datos.

  Algunos ejemplos de lenguajes de programación son **Haskell, Miranda y Scheme.**

  La programación lógica, junto con la funcional, forman parte de lo que se conoce como **PARADIGMA DECLARATIVO**, es decir la programación consiste en indicar como resolver un problema mediante sentencias. Se basa en el desarrollo de programas, pero especificando o "declarando" un conjunto de condiciones, proposiciones, afirmaciones, restricciones, ecuaciones o transformaciones que describen el problema y detallan su solución. La solución es obtenida mediante mecanismos internos de control, sin especificar exactamente cómo encontrarla, Algunos ejemplos son **Prolog y Lisp**.

- **IMPERATIVO:** en oposición a la programación declarativa, este es un paradigma de programación que describe - valga la redundancia - la programación, en términos del estado del programa y las sentencias que cambian dicho estado.

  Es la forma natural de programar un computador, es el estilo de programación que se utiliza en el ensamblador, el estilo más cercano a la arquitectura del computador, esta después del Lenguaje máquina, o lenguaje de bajo nivel.

  De forma más sencilla, la programación imperativa es la más común y conocida. Pues son un conjunto de instrucciones que le indican a la computadora cómo realizar una tarea.

Está representada por lenguajes de programación como **C, Basic, Fortran, etc**

- **ESTRUCTURADO:** es una teoría de programación que consiste en construir programas de fácil comprensión, este paradigma es útil, cuando se necesitan realizar correcciones o modificaciones después de haber concluido un programa.

  Al utilizar la programación estructurada, es mucho más sencillo entender la codificación del programa, que se habrá hecho en diferentes secciones.

  Los diagramas de flujo se convierten en la representación gráfica de la programación estructurada, en función de entradas, procesos y salidas.

  Algunos ejemplos son: **ADA, Pascal, C/C++**

- **MODULAR:** surge el paradigma de la programación modular consiste en: establecer los módulos que se requieren para la resolución de un problema, dividir el programa de modo que los procedimientos y los datos queden ocultos en módulos.

  Este paradigma también es conocido como un principio de ocultación de procedimientos y datos.

  Un ejemplo puede ser **C++** que aunque no se diseñó específicamente para desarrollar la programación modular, su concepto de clase proporciona apoyo para la noción de módulo.

- **ORIENTADO A OBJETOS (POO):** Este tipo de programación hace parte de la programación imperativa.

  Esta intenta simular el mundo real a través del significado de objetos que contiene características y funciones. Se basa en la idea de un objeto, que es una combinación de variables locales y procedimientos llamados métodos que juntos conforman una entidad de programación. En la actualidad, existe una gran variedad de lenguajes de programación que soportan la orientación a objetos, tales como: **C++, Java, Smalltalk, Ruby on Rails, Python y JavaScript,** entre muchos otros.

¿Qué es la programación orientada a Objetos?

(Corporation, s.f.) El término Programación Orientada a Objetos (POO), hoy en día ampliamente utilizado, es difícil de definir, ya que no es un concepto nuevo, sino que ha sido el desarrollo de técnicas de programación desde principios de la década de los setenta, aunque sea en la década de los noventa cuando ha aumentado su difusión, uso y popularidad. No obstante, se puede definir POO como una técnica o estilo de programación que utiliza objetos como bloque esencial de construcción.

Nuestro alrededor está formado por objetos de diversa índole: Unos son naturales, como los seres vivos, animales y plantas; otros como los del mundo mineral; los hay también artificiales como los construidos por el ser humano o por máquinas concebidas y elaboradas por él.

¿Qué es un Objeto?

Un objeto en la programación orientada a objetos (POO) es una estructura de datos y conjunto de procedimientos que operan sobre dicha estructura.

Una definición más completa de objeto es: una entidad de programa que consiste en datos y todos aquellos procedimientos que pueden manipular aquellos datos; el acceso a los datos de un objeto es solamente a través de estos procedimientos. A los elementos de un objeto se les conoce como miembros; las funciones que operan sobre los objetos se denominan métodos y los datos se denominan miembros datos.

Un ejemplo sencillo para entender lo que es un objeto seria:

- Un **Gato**: tiene cola, patas, pelo, hocico, bigotes, etc. Como también Maulla, come ratones, juega, aruña etc.
- **Computador**: tiene teclado, mouse, chips, pantalla, disco duro, memoria, procesador etc. Este puede, procesar datos, hacer cálculos, organizar, estadísticas, entre otras funciones.

Por donde miremos vemos objetos que tienen características propias y comportamientos únicos.

Regla de Denominación de un objeto: Para la designación de nombres de objetos o identificadores de nombres de objetos, se sigue la norma de utilizar nombres que sean significativos, y que describan al objeto en sí, con la única condición que la primera palabra es toda con minúsculas y las siguientes palabras sin espacio entre ellas, se inician con mayúscula, las demás letras son minúsculas.

Ejemplo: cajaDeColores, sabana… etc.

¿Qué es una clase?

Podemos considerar una clase como una colección de objetos que poseen características y operaciones comunes. Una clase contiene toda la información necesaria para crear nuevos objetos.

Es un ente que se usa para modelar un problema de la realidad, caracterizado por unos atributos (propiedades) y unos métodos (acciones) que le permiten resolver dicho problema.

Los nombres para denominar las clases siguen la misma norma que los objetos, con la condición que la primera palabra se inicia con mayúscula.

Ejemplo: Trabajador, EquipoDeTrabajo, Carro, Persona… etc.

Es debido anotar que es el problema quien genera la clase y no lo contrario, dado que de las particularidades del problema y de su contexto mismo es de donde se abstraen las propiedades (atributos) y métodos (acciones) de la clase.

Donde **Los atributos** serán todas aquellas características que tendrá el objeto y que lo definirán en su totalidad, pues caracteriza el contenido como el valor cambiante que depende de las operaciones que se hagan en él, pero el identificador que se le asigno no cambia nunca.

Para poder acceder al contenido del atributo hay que hacer referencia a su identificador. Hay que tener en cuenta al momento de darle un identificador (nombre) a un atributo es recomendable relacionarlo con el concepto que queremos referenciar (mnemotécnica).

Para referenciarlos y darles nombres a los atributos debemos tener en cuenta que estos comienzan con la letra minúscula, no deben comenzar con números, no deben ser palabras reservadas del lenguaje de programación, no deben contener espacios, por cuestiones prácticas deben ser claros y no tan largos y deben ser mnemotécnicos o sea deben caracterizar al dato que manipularan.

Estos atributos gozaran de pertenecer a un tipo de dato, es decir, solo podrá ser un solo tipo de dato a la vez. Existen 3 tipos de datos:

- **Numéricos:** dentro de los numéricos tenemos dos tipos:

    - **Tipo Entero:** Los tipos enteros se utilizan para representar valores enteros con signo, esto es, pueden tomar valores tanto positivos como negativos. Los diferentes tipos de enteros se diferencian en el rango de valores que pueden tomar. La tabla siguiente muestra los tipos de enteros y su rango.

Tabla 1 Variables Numéricas (Tipo Entero)

Tipo	Especificador de tipo	Tamaño (en bytes)	Rango
Byte	byte	1	-128 a 127
Entero corto	short	2	-32768 a 32767
Entero	int	4	-2.147.483.648 a 2.147.483.647
Entero largo	long	8	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

Ejemplo: la talla de una persona, la edad de una persona, la suma de dos números enteros (5+5=10), entre otros.

- **Tipo Reales:** Los tipos reales o de punto flotante se utilizan cuando deseamos trabajar con números que tienen parte fraccionaria. La siguiente tabla muestra los tipos reales y su rango.

Tabla 2 Variables Numéricas (Tipo Reales)

Tipo	Especificador de tipo	Tamaño (en bytes)	Rango	Precisión (en cifras significativas)
Flotante	float	4	$3.4e^{-38}$ a $3.4e^{+38}$	7
Doble	double	8	$1.7e^{-308}$ a $1.7e^{+308}$	15

Ejemplo: el resultado de una división (5/2= 2.5), el diámetro de un circulo, etc.

- **Alfanuméricos (todo tipo de caracter):** tenemos dos tipos

  - **Tipo Caracter:** nos permite almacenar caracteres unitarios y su tamaño es de dos bytes.

Tabla 3 Variables Alfanuméricas (Tipo Carácter)

Tipo	Especificador de tipo	Tamaño (en bytes)	Rango
Caracter	char	2	0 a 65535

Ejemplo: el sexo de una persona (F/M); una letra, o un numero de un digito.

  - **Tipo Cadena:** permite almacenar caracteres alfanuméricos del tamaño que se requiera.

Tabla 4 Variables Alfanuméricas (Tipo Cadena)

Palabra Reservada	Rango de Valores
String	Cadena de caracteres según usuario

Ejemplo: Nombres completos, apellidos, correo, oraciones, números de identificación, textos, entre otros.

- **Booleanos (valores lógicos 0 y 1; True, False):**

Tabla 5 Variables Booleanas

Palabra Reservada	Rango de Valores
boolean	Valor lógico: True o False

Ejemplo: verificación de estado (activo e inactivo), verificaciones de procesos, entro otros.

Entonces si aplicamos el concepto de tipos de datos sobre los atributos , debemos definirlos indicando el tipo de datos que va a manipular de la siguiente manera:

```
String direccion;
String nombreCompleto;
int edad, peso;
boolean estado;
char sexo;
float talla;
```

Los métodos son las acciones que debe ejecutar la clase para resolver el problema (manipulando los datos con las operaciones necesarias).

Cada método debe realizar una tarea específica de acuerdo al problema que se quiere solucionar con la clase en mención.

Existen 4 tipos de métodos:

- **CONSTRUCTORES:** Se compone de todas las operaciones de inicialización de valores para los atributos, como también la inicialización de las estructuras internas de las clases (direccionamiento de memoria), por lo tanto deben ser los primeros en activar al momento de utilizar la clase.

  Este método tiene las siguientes características:

  - Posee igual nombre que la clase a la que pertenece.
  - No se hereda.
  - No devuelve ningún valor.
  - Debe declararse público para que pueda ser invocado desde cualquier parte donde se desee crear un objeto de su clase.

  Sintaxis: Palabra reservada: nombre_Clase ()
  Ejemplo: multiplicar();

- **DESTRUCTORES:** Tienen como funcionalidad liberar los recursos de memoria utilizados por la clase para su trabajo. Su activación implica que el objeto no podrá ser usado para tareas posteriores en el ámbito del programa que esté haciendo uso de él.

  Otra característica de los destructores en el lenguaje java es que no es necesario declararlos.

  Sintaxis:

  Palabra reservada : finalize()
  Ejemplo: protected void finalize()

- **PROCEDIMIENTOS:** Caracterizan por la ejecución de una o más acciones (operaciones) sobre unos datos o valores que tienen un tipo (dominio) determinado y que se obtienen en algunos casos leyéndose de los atributos, tomándose a partir de parámetros (concepto que definiremos más adelante) o utilizando otros métodos de la clase.

  Sintaxis:

  Palabra reservada : void Identificador;
  Ejemplo: void pedir_Numeros( );

- **FUNCIONES:** Al igual que los procedimientos, una función constituye un método que realiza operaciones sobre datos de un tipo específico; datos que son leídos desde los atributos, otros métodos de la clase y/o pasados por parámetros, pero que a diferencia de los procedimientos, las operaciones efectuadas sobre dichos datos devuelven uno y solo un valor que por supuesto tiene un tipo definido y el cual caracteriza el tipo de la función.

  Sintaxis:

  Palabra reservada : Tipo de dato Identificador;
  Ejemplo: float multiplicacion( );

Ejercicio: Se quiere diseñar una clase que permita calcular el área de un trapecio.

Solución

Lo primero que debemos tener en cuenta y saber son los datos que vamos a requerir para poder calcular el área del trapecio; por lo tanto hay que recordar que:

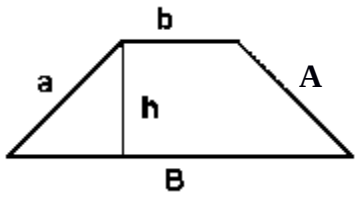


Ilustración 1 Trapecio

El área de un trapecio es igual a la suma de sus dos bases por la altura, todo dividido entre dos; y
El perímetro es igual a la suma de las bases y de los lados.
Si vemos la imagen, podemos notar que las bases están denominadas como "B" Y "b" siendo la primera la base mayor y la segunda la base menor.

Los lados del trapecio están designados con las letras "A" y "a" para nombrar respectivamente el lado mayor y el lado menor del trapecio. Por último la "h" es el valor de la altura o distancia entre las dos bases.

Ahora, teniendo cuenta toda esa información se procede a caracterizar los atributos y los metodos que harán parte de este problema.

ATRIBUTOS: para poder resolver estos interrogantes es necesario tener dos atributos **Base** y un atributo **Altura**; así como también dos atributos **Lado.** Por lo tanto mis atributos serán:

```
float baseMayor;
float baseMenor;
float altura;
float ladoMayor;
floar ladoMenor;
```

METODOS: como primer paso debemos crear un método que se encargue de guardar en los atributos los valores correspondientes que se pedirán, a este método lo llamaremos pedir_Valores.

Luego debemos crear un método obtener_Valores es el que nos devuelve los datos que se han almacenado.

Ahora para CALCULAR EL AREA es necesario un método que se encargara de hacer la operación de (sumar las bases, multiplicar por la altura y dividir el resultado entre dos); este método lo llamaremos calcular_Area.

Después para calcular el PERIMETRO se creara un método llamado calcular_Perimetro que será el encargado de sumar los valores de las bases y los valores de los lados del trapecio.

Para mostrar el resultado de las operaciones que realiza la clase creamos el método mostrar_Resultados.

Por tanto los métodos creados para resolver nuestro problema serán:

```
pedir_Valores();
obtener_Valores();
calcular_Area();
calcular_Perimetro();
mostrar_Resultados();
```

Nota: los paréntesis que se colocan seguidos del nombre del método indican el espacio donde se declararan los atributos o parámetros que necesitara el método si son necesarios.

PARAMETROS

Para que los métodos puedan ejecutarse deberán tomar valores ya sea a la clase misma o a elementos externos.

Cuando un método necesita valores que son externos y estos valores pueden variar al momento de ser manipulados en los procesos mismos del método, estos valores reciben el nombre de parámetros.

Nota: Cada vez que necesitamos valores cambiantes en un método y no conocemos su contenido, debemos hacer la referencia en el método con un parámetro.

Tipos de Parámetros:

- **Parámetros por valor:** Cuando el parámetro es utilizado solamente para introducir valores al método que lo procesa.

- **Parámetros por referencia:** Cuando el parámetro al ser asignada al método puede cambiar o modificar su valor durante su utilización en el método, manteniendo esos cambios de allí en adelante para dar a conocer dicho cambio a los demás miembros de la clase, quienes pueden utilizarlo y modificar de nuevo dicho valor.

ENCAPSULADO

La encapsulación es un instrumento que consiste en organizar atributos y métodos de una estructura, evitando el acceso a datos por cualquier otro medio distinto a los especificados, es decir, la encapsulación garantiza la integridad de los datos que contiene un objeto.

El encapsulamiento define dos reglas

1. **Sobre los atributos**: para acceder a ellos (asignarles valores o leer los mismos), se deben definir métodos especializados en dichas operaciones de tal manera que los atributos son manejados por medio de la activación de dichos métodos.

2. **Sobre los métodos:** criterio de unicidad funcional, es decir, que un método debe encargarse de realizar una tarea (proceso) única; De esta forma una clase define tantos métodos como tareas debe efectuar.

Como su esencia es la de proteger los datos con la utilización de métodos, se deben diferenciar diversas secciones dentro de una clase.

- **Sección Privada:** En esta sección se declaran los atributos y métodos que solo pueden ser accedida por los métodos de la misma clase, los descendientes no tienen acceso a ellos. En esta sección antes de los atributos se debe colocar el signo menos "-" para mostrar que dicho atributo es de tipo privado.
- **Sección Protegida**: se declaran los atributos y métodos de la clase que pueden ser accedidos por sus propios métodos y por los métodos de sus descendientes
- **Sección Pública:** Se declaran los métodos que la clase a poner a disposición del usuario. En esta sección antes de los métodos se debe colocar el signo más "+" para mostrar el método es de tipo público.

La forma gráfica en la que se presenta una clase teniendo en cuenta el encapsulamiento es una tabla que consta de 1 columna y 4 filas, como se ve a continuación:

Clase
Sección Privada -(atributos, métodos)
Sección Protegida (atributos y métodos)
Sección Publica +(métodos)

Ilustración 2 Diseño de Encapsulado de una clase

Y la sintaxis del encapsulamiento es la siguiente:

```
Class Identificador_Clase
   {
      private Tipo_dato  identificadorAtributo;
      public  Tipo_dato  identificador_Metodo() {

              //implementación del método

              }
    };
```

Diseño UML

(Corporation, Curso de Java POO: Programación orientada a objetos, s.f.) UML (Unified Modeling Language): Lenguaje unificado de modelos

Es un "Mapa" del código. No sirve para desarrollar, sino para describir.

Si retomamos el ejercicio realizado sobre el área y el perímetro de un trapecio, podemos ejemplificar como se realiza un diseño UML

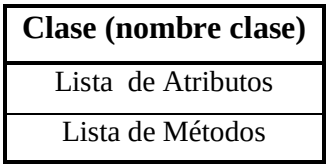
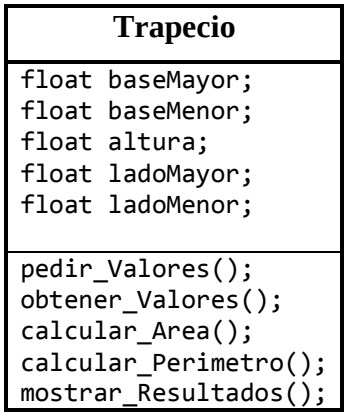


Ilustración 3 Diseño UML

Así, tenemos que para el ejercicio nuestro diseño UML quedaría de la siguiente manera



Diseño de una clase

La representación de una clase es un diseño UML: En donde la primera fila contendrá el nombre de la clase, la segunda los atributos propias o privados de la clase y la tercera y última tendrán los métodos públicos de la clase.

Para ejemplo seguiremos tomando el ejercicio anterior (área y perímetro del trapecio), teniendo en cuenta su diseño de encapsulado.

Trapecio
-float baseMayor; -float baseMenor; -float altura; -float ladoMayor; -float ladoMenor;
+void setpedir_baseMayor(float B); +void setpedir_baseMenor(float b); +void setpedir_altura(float h); +void setpedir_ladoMayor(float A); +void setpedir_ladoMenor(float a; +float getbaseMayor(); +float getbaseMenor(); +float getaltura(); +float getladoMayor(); +float getladoMenor(); +float calcular_Area(); +float calcular_Perimetro(); +float mostrar_Resultados();

Estructura de una clase en JAVA

Continuando con el ejercicio anterior podemos pasar del diseño de una clase a su estructura en lenguaje de programación JAVA.

```
Class  Trapecio {

  private
         float  baseMayor, baseMenor, altura, ladoMayor, ladoMenor;

  public
         trapecio ();
         void  pedir_Valores (float valorbaseMayor;  float valorbaseMenor,
         float valoraltura, float valorladoMayor, float valorladoMenor);
         float obtener_baseMayor();
         float obtener_baseMenor();
```

```
        float obtener_altura();
        float obtener_ladoMayor();
        float obtener_ladoMenor();
        float calcular_Area();
        float calcular_Perimetro();
        void  mostrar_Resultados();
};
```

En la estructura definida en la clase notamos varios aspectos de importancia:

- Existen 5 atributos (2 bases, 2 lados y una altura) que como se definió deben poseer un dominio (tipo de dato) que para el caso es float, la razón para ello es que deben ser valores tipo real puesto que dentro de las operaciones que se van a realizar debemos hacer una división y en caso de que esta no de un numero entero, se debe mostrar también la parte decimal.
- Los 5 atributos se encuentran en la sección privada atendiendo a la regla del encapsulamiento que nos dice que debemos proteger todos los atributos.
- En la sección pública tenemos que: trabajamos con un constructor al cual le llamamos como la clase "trapecio", se diseñan tres métodos para acceder a los atributos: para llevar información a los atributos diseñamos el método **"**pedir_Valores" el cual utilizan cinco parámetros (valorbaseMayor, valorbaseMenor, valoraltura, valorladoMayor, valorladoMenor) los cuales van dentro de los paréntesis del método; y cinco para traer información de los atributos que son: "obtener_baseMayor()", "obtener_baseMenor()", "obtener_altura()", "obtener_ladoMayor()", "obtener_ladoMenor()".
- Seguidamente se diseñan los métodos que han de ser las acciones para lo cual se ha diseñado la clase, siendo éstos "calcular_Area" y "calcular_Perimetro", además un método para mostrar resultados (mostrar_Resultados).

OBSERVACION: Cabe hacer referencia en la sintaxis del lenguaje java: la definición de los métodos en java implica la implementación del código de los mismos dentro de la clase, algo que omitimos ahora pero más adelante atenderemos y aprenderemos.

EJERCICIOS/ACTIVIDADES

1. Investigue y Realice un cuadro comparativo sobre los paradigmas de programación donde quede evidenciado las similitudes y las diferencias.

2.Indique si las siguientes afirmaciones son verdaderas o falsas

a. La programación orientada a objetos hace uso de clases y envío de mensajes entre objetos. .V– F
b. Las clases están compuestas por métodos y atributos .....V– F
c. El constructor predefinido retorna un tipo void…….….. V– F
d. El constructor predefinido toma como parámetro un valor tipo void……..V– F
e. El constructor predefinido toma parámetros. ……. …… …….V– F

3. ¿Cuál de las siguientes características no pertenece a un objeto?

a. Un objeto es identificable por un único nombre
b. Un objeto tiene estados.
c. Un objeto puede formar parte de una clase.
d. Un objeto puede pertenecer a distintas clases.
e. Todas las anteriores 3.

4. Los atributos de un objeto forman:

a. Los valores posibles del objeto.
b. Su estado.
c. Su comportamiento.
d. Su conjunto de métodos.

5. Detallar mínimo 5 atributos para las siguientes clases:

i. Persona
ii. Animal
iii. Vehículo
iv. Inmueble
v. Estudiante
vi. Reserva de vuelo
vii. Entrada cine
Detallar mínimo 5 métodos para cada una de las clases, especificando que haría cada método.

BIBLIOGRAFIA

Mestras, J. P. (Agosto de 2007). 1.1.Objetos y Clases. Obtenido de https://www.fdi.ucm.es/profesor/jpavon/poo/1.1.Objetos%20y%20Clases.pdf

Mtra. Ivonne Robledo, M. A. (s.f.). Metodologias. Obtenido de http://www.uacj.mx/CGTI/CDTE/JPM/Documents/IIT/proceprogra/metodologias.html

Mtra. Ivonne Robledo, M. A. (s.f.). Solucionar Problemas Usando Algoritmos. Obtenido de http://www.uacj.mx/CGTI/CDTE/JPM/Documents/IIT/proceprogra/solucionar-problemas-usando-algoritmos.html

Rodríguez, A. (s.f.). Métodos en Java tipo procedimiento (…void) y tipo función (…return). Definición. Invocación. (CU00624B).

Corporation, M. (s.f.). Curso de Java POO: Programación orientada a objetos. Obtenido de http://mmc.geofisica.unam.mx/femp/ProgramacionOrientadaObjetos/POO_1.pdf

Corporation, M. (s.f.). Microsoft Word - cap1.doc. Obtenido de http://mmc.geofisica.unam.mx/femp/ProgramacionOrientadaObjetos/POO_3.pdf

http://www.iqcelaya.itc.mx/~vicente/Programacion/Paradigmas.pdf

https://dokupdf.com/download/paradigmas-_5a01961fd64ab2b9bd5721e7_pdf

http://puntocomnoesunlenguaje.blogspot.com.co/2012/07/normal-0-21-false-false-false_103.html

https://es.ccm.net/contents/410-poo-encapsulacion-de-datos

https://www.unioviedo.es/ate/alberto/TEMA3-Ensamblador.pdf